

ภาคผนวก ก

โปรแกรมที่สมบูรณ์ของ Crane Controller

```

/*
 * crane_controller program base on liftmon_snowcon program
 * of embedded linux book examples
 * modified for embedded linux project on 5/5/04
 *
 * this program used for control crane simulator set
 */

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/proc_fs.h>
#include <linux/sched.h>
#include <asm/uaccess.h>
#include <linux/ioport.h>
#include <linux/interrupt.h>
#include <asm/io.h>

#define MODULE_VERSION "1.0"
#define MODULE_NAME "crane_controller"

/* All SPP PORT definitions */
#define SPPDATAPORT 0x378
#define SPPSTATUSPORT (SPPDATAPORT + 1)
#define SPPCONTROLPORT (SPPDATAPORT + 2)
#define INTERRUPT 7

/* SPP control port bit definitions */
#define OUTPUTENABLE 0x02
#define OUTPUTLATCH 0x04

```

```

#define INPUTENABLE                0x08
#define SPPPORTREAD                0x20
#define SPPINTERRUPTENABLE        0x10

#define MAXSTEP 3 /*max index of step table */

/* all delay time use in jiffies style */
#define MAXMOTORDELAY 1 /*used for delay motor driving */
#define MAXSENSORDELAY 200 /*used for delay x,y sensor input not occur within */
#define MINSSENSORDELAY 150 /*used for delay x,y sensor input not occur within */
#define MAXINTERRUPTDELAY 1
#define MAXTIMEOUTDELAY 1500

/* SPP input bit definitions */
#define BOTTOMSW                0x01                /* bottom limit switch */
#define TOPSW                  0x02                /* top limit switch */
#define LEFTSW                 0x04                /* left limit switch */
#define RIGHTSW                0x08                /* right limit switch */
#define XPOSSENSOR             0x10                /* crane x position */
#define YPOSSENSOR             0x20                /* crane y position */

/* SPP output bit definitions */
#define XMOTORMASK              0x0F
#define YMOTORMASK              0xF0

MODULE_LICENSE("GPL");

/* These are the pointers to the /proc directory entries */
static struct proc_dir_entry    *main_dir,        /* main process directory */
*move_only_file                /* move motor only process */
*get_position_file,            /* read crane position process */
*get_status_file;              /* read all crane status process */

```

```

/*****

all status messages

*****/

/* crane status */

char *init_string = "Initializing crane please wait\n";
char *standby_string = "Crane standby for your request\n";
char *moving_string = "Crane moving to specified position\n";
char *demo_string = "Program running in demo mode\n";
char *timeout_error_string = "Crane not reach to target position in time\n";
char *left_limit_string = "Left limit action\n";
char *right_limit_string = "Right limit action\n";
char *top_limit_string = "Top limit action\n";
char *bottom_limit_string = "Bottom limit action\n";

/* user warning message */

char *user_position_error = "Please enter X,Y position between 0,0 and 5,5 coordinate\n";
char *user_keyin_error = "Please enter number in format X,Y between 0,0 and 5,5 coordinate\n";

/*****

/* define global variables here */

*****/

int xyposchange=1; /*used for protect x,y position if motor not moving */
int timeout = 0;      /*used for limit motor action */
long xmotorstarttime = 0; /* used for check x sensor interval time */
long ymotorstarttime = 0;
int xflag = 0;        /* start x sensor flag */
int yflag = 0;
int xflag2 = 0;
int yflag2 = 0;
int xpos=0;           /*crane x position counter */
int ypos=0;           /*crane y position counter */

char xdirect,ydirect; /* global variable for motor direction */
char progmode = 'a';  /* a = actual , d = demo */

```

```

unsigned short motorout ;           /*variable for keep stepping motor data out */

unsigned char ystepable[MAXSTEP+1] = {0x03,0x06,0x0c,0x09};
unsigned char xstepable[MAXSTEP+1] = {0x90,0xc0,0x60,0x30};

int xindex=0;                       /*stepper motor move index */
int yindex=0;

/******

/* sub program header define */

/******

void move_to_00(void);
void CheckInputAndAction(void);
void move_x_motor(void);
void move_y_motor(void);
void send_outport(void);
void XPosAction(void);
void YPosAction(void);

/******

/* common use function definind */

/******

void mydelay(long mytimer)
{
    long mydelaytimer;
    mydelaytimer = jiffies;
    while((jiffies - mydelaytimer) < mytimer) ;
}

/* use for change user char input to integer for use in comparison */
int chartoint(int charin)
{
    int retval;
    switch(charin)
    {
        case '0' : retval = 0; break;

```

```

        case '1' : retval = 1; break;
        case '2' : retval = 2; break;
        case '3' : retval = 3; break;
        case '4' : retval = 4; break;
        case '5' : retval = 5; break;
        /*case '6' : retval = 6; break;
        case '7' : retval = 7; break;
        case '8' : retval = 8; break;
        case '9' : retval = 9; break;*/
        default: retval = -1;

    }

    return retval;
}

/* use for move motor in 3 direction */
/* 'r' = reverse direction */
/* 'f' = forward direction */
/* 'n' = none no move */
/* x motor move */
void move_x_motor()
{
    if(xdirect == 'r')
    {
        xindex--;                      /* move x motor backward */
        if(xindex < 0) xindex = MAXSTEP;
    }
    else if(xdirect == 'f')
    {
        xindex++;                      /* move x motor forward */
        if(xindex > MAXSTEP) xindex = 0;
    }
}

```

```

/* y motor move */
/* void move_y_motor(char direction) */
void move_y_motor()
{
    if(ydirect == 'r')
    {
        yindex--;          /* move y motor backward */
        if(yindex < 0) yindex = MAXSTEP;
    }
    else if(ydirect == 'f')
    {
        yindex++;          /* move y motor forward */
        if(yindex > MAXSTEP) yindex = 0;
    }
}

void send_outport()
{
    motorout = 0;
    motorout = ystepable[yindex] | xstepable[xindex];
    outb((motorout & 0xFF) , SPPDATAPORT);
}

/*proc_read_position
* this process used for send crane position to user when user read position_file
*/

static int proc_read_position(char *page,char **start, off_t off, int count, int *eof, void *data)
{
    page[0] = (char) (xpos + 48);
    page[1] = (char) (ypos + 48);
    page[2] = '\n';
    return 3;
}

```

```

/*proc_read_status
* this process used for send crane status to user when user read status_file
*/

static int proc_read_status(char *page,char **start, off_t off, int count, int *eof, void *data)
{
    char *outchar = (char *)data;
    int i=0;
    while((outchar[i] != '\n') && (i<255))
    {
        page[i] = outchar[i];
        i++;
    }
    page[i] = '\n';
    i++;
    return i;
}

/*proc_move_only
* this process used for recieve input x,y position from user and
* move crane to specified position only
*/

static int proc_move_only(struct file *file, const char *buffer,
                          unsigned long count, void *data)
{
    long i;
    int temp_x,temp_y;
    /*char xdirect,ydirect; */
    /* check data input correct */
    if(count > 3)
    {
        printk("user request position more than 2 digit\n");
        get_status_file->data = user_position_error; /* set status text to user mistake */
    }
}

```

```

        goto nothing_to_do;
    }
    else if(count < 3)
    {
        printk("user request position less than 2 digit\n");
        get_status_file->data = user_position_error; /* set status text to user mistake */
        goto nothing_to_do;
    }
    else printk("move to x=%c y=%c\n",buffer[0],buffer[1]);

    /*change char to int for compare*/
    tempx = chartoint(buffer[0]);          /*set x target to tempx */
    tempy = chartoint(buffer[1]);          /*set y target to tempy */

    /* check for valid number */
    if((tempx==-1) || (tempy==-1))
    {
        printk("user not keyin number\n");
        get_status_file->data = user_keyin_error; /* set status text to user mistake */
    }
    else /* every thing ok so we move the motor */
    {
        printk("x and y motor moving\n");
        get_status_file->data = moving_string ;
        if((tempx==0)&&(tempy==0)) move_to_00();
        else
        {
            timeout =0;
            xyposchange = 0;
            while(!((tempx == xpos) && (tempy == ypos))&&(timeout <
                MAXTIMEOUTDELAY))/* wait for x and y position */
            {
                /* check x position for move */
                if(tempx < xpos)

```



```

        xdirect = 'r';    /* move reverse direction */
    else if(tempx > xpos)
        xdirect = 'f';    /* move forward direction */
    else
        xdirect = 'n';    /* not move */

    /* check y position for move */
    if(tempy < ypos)
        ydirect = 'r';    /* move reverse direction */
    else if(tempy > ypos)
        ydirect = 'f';    /* move forward direction */
    else
        ydirect = 'n';    /* not move */

    move_x_motor();
    move_y_motor();
    send_outport();
    mydelay(MAXMOTORDELAY);
    timeout ++;
}
printf("crane stand by \n");
get_status_file->data = standby_string ;
}
}

if(timeout >= MAXTIMEOUTDELAY)
{
    get_status_file->data = timeout_error_string ;
    move_to_00();
}

nothing_to_do:
xyposchange = 1;
return count;
}

```

```

void XPosAction()
{
    if((xyposchange==0))          /* not count if motor not moving */
    {
        if(xdirect == 'r')
            xpos -- ;
        if(xdirect == 'f')
            xpos ++;
    }
}

void YPosAction()
{
    if(xyposchange==0)
    {
        if(ydirect == 'r')
            ypos -- ;
        if(ydirect == 'f')
            ypos ++;
    }
}

void CheckInputAndAction()
{
    int ipA;
    outb(SPPPORTREAD|INPUTENABLE,SPPCONTROLPORT);
    ipA = inb(SPPDATAPORT);
    if(ipA & 0x80)                /*bottom limit switch */
    {
        ydirect = 'n';
        ypos = 0;
    }
}

```

```

if(ipA & 0x20)          /* top limit switch */
{
    ydirect = 'n';
    ypos = 6;
}

if(ipA & 0x40)          /*left limit switch */
{
    xdirect = 'n';
    xpos = 0;
}

if(ipA & 0x10)          /* right limit switch */
{
    xdirect = 'n';
    xpos = 6;
}

if(ipA & 0x01)          /* x sensor */
{
    if(xyposchange==0)
    {
        if(xpos==0)
        {
            if(xflag==0)
            {
                xflag = 1;
                xmotorstarttime = jiffies;
                XPosAction();
                printk("x sensor active  jiffies=%d \n",jiffies);
            }
            else
            {
                if((jiffies-xmotorstarttime) > MAXSENSORDELAY)

```



```

{
    if(ypos==0)
    {
        if(yflag==0)
        {
            yflag = 1;
            ymotorstarttime = jiffies;
            YPosAction();
            printk("y sensor active  jiffies=%d \n",jiffies);
        }
        else
        {
            if((jiffies-ymotorstarttime) > MAXSENSORDELAY)
            yflag = 0;
        }
    }
    else
    {
        if(yflag==0)
        {
            yflag = 1;
            ymotorstarttime = jiffies;
        }
        else
        {
            if(((jiffies-ymotorstarttime) > MINSSENSORDELAY) &&
                (yflag2==0))
            {
                yflag2 = 1;
                YPosAction();
                printk("y sensor active  jiffies=%d \n",jiffies);
            }
        }
    }
}

```

```

        if(((jiffies-ymotorstarttime) > MAXSENSORDELAY) &&
            (yflag2==1))
        {
            yflag2 = 0;
            yflag = 0;
        }
    }
}

outb(SPPINTERRUPTENABLE|OUTPUTENABLE|OUTPUTLATCH,SPPCONTROLPORT);
}

void move_to_00(void)
{
    long i;

    /* initial crane position to -1 */
    xdirect = 'r';
    ydirect = 'r';
    timeout = 0;
    CheckInputAndAction();
    while(((xdirect != 'n') || (ydirect != 'n')) && (timeout < MAXTIMEOUTDELAY) )
    {
        move_x_motor();
        move_y_motor();
        send_outport();
        mydelay(MAXMOTORDELAY);
        timeout ++;
    }
}

```

```

/*function crane_interrupt_interrupt
This function is used for action when interrupt occur */

void crane_interrupt_interrupt(int irq, void *dev_id, struct pt_regs *regs)
{
    CheckInputAndAction();
}

/* init - init_crane_controller */
static int __init init_crane_controller(void)
{
    int rv = 0;
    int i;

    /* Create the crane_controller /proc entry */

    main_dir = proc_mkdir("crane_controller", NULL);
    if(main_dir == NULL)
    {
        rv = -ENOMEM;
        goto out;
    }

    main_dir->owner = THIS_MODULE;

    /* Create liftacmains and make it readable by all - 0444 */

    move_only_file = create_proc_entry("move_only", 0666, main_dir);
    if(move_only_file == NULL)
    {
        rv = -ENOMEM;
        goto no_move_only;
    }

    move_only_file->read_proc = NULL;
    move_only_file->write_proc = &proc_move_only;
    move_only_file->owner = THIS_MODULE;

```

```

/*status file */

get_status_file = create_proc_entry("crane_status", 0444, main_dir);
if(get_status_file == NULL)
{
    rv = -ENOMEM;
    goto no_get_status;
}

get_status_file->data = init_string;
get_status_file->read_proc = &proc_read_status;
get_status_file->write_proc = NULL;
get_status_file->owner = THIS_MODULE;

/*position file */

get_position_file = create_proc_entry("crane_position", 0444, main_dir);
if(get_position_file == NULL)
{
    rv = -ENOMEM;
    goto no_get_position;
}

get_position_file->read_proc = &proc_read_position;
get_position_file->write_proc = NULL;
get_position_file->owner = THIS_MODULE;

crane_interrupt_file = create_proc_entry("crane_interrupt", 0444, main_dir);
if(crane_interrupt_file == NULL)
{
    return -ENOMEM;
}

crane_interrupt_file->read_proc = NULL;
crane_interrupt_file->write_proc = NULL;
crane_interrupt_file->owner = THIS_MODULE;

```



```

/* request interrupt from linux */
rv = request_irq(INTERRUPT, crane_interrupt_interrupt, 0,"crane_interrupt",NULL);
if ( rv )
{
    printk("Can't get interrupt %d\n", INTERRUPT);
    goto no_interrupt;
}

move_to_00();
if(timeout >= MAXTIMEOUTDELAY)
{
    printk("timeout = %d , xdirect = %c , ydirect = %c \n",timeout,xdirect,ydirect);
    printk("Could not detect crane controller board. Program running in demo mode\n");
    get_status_file->data = demo_string;
}
else
{
    printk("timeout = %d , xdirect = %c , ydirect = %c \n",timeout,xdirect,ydirect);
    printk("Program running in real time mode\n");
    get_status_file->data = standby_string ;
}

/* everything initialed */
printk(KERN_INFO "%s %s initialized\n", MODULE_NAME, MODULE_VERSION);
return 0;

/* this removes /proc entries if we have an error along the way */
no_interrupt:
    remove_proc_entry("crane_interrupt",main_dir);
no_get_position:
    remove_proc_entry("crane_position",main_dir);
no_get_status:
    remove_proc_entry("crane_status",main_dir);

```

```

        no_move_only:

            remove_proc_entry("move_only", main_dir);

        out:

            return rv;
    }

/* exit - cleanup_crane_controller */
static void __exit cleanup_crane_controller(void)
{
    free_irq(INTERRUPT, NULL);

    /* removing the /proc entries */
    remove_proc_entry("crane_interrupt", main_dir);
    remove_proc_entry("crane_position", main_dir);
    remove_proc_entry("crane_status", main_dir);
    remove_proc_entry("move_only", main_dir);
    remove_proc_entry("crane_controller", NULL);

    /* we're done */
    printk(KERN_INFO "%s %s removed\n", MODULE_NAME, MODULE_VERSION);
}

module_init(init_crane_controller);
module_exit(cleanup_crane_controller);

MODULE_AUTHOR("Sthaporn Sahounwong");  MODULE_DESCRIPTION("Crane controller
process module");  EXPORT_NO_SYMBOLS;

```